API Automation Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**299**
Exact duplicates removed (same question, different wording of punctuation or case)	**7**
Near-duplicates merged (same question asked in different words)	**171**
Unique questions in this PDF	**121**

Section 1: Critical (Must Prepare)	33 questions
Section 2: High Priority	57 questions
Section 3: Medium Priority	28 questions
Section 4: Low Priority / Quick Revision	3 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 33 questions	Asked in almost every API automation interview at 4+ years. Be ready to explain the concept and to write the REST Assured code or Postman script on the spot.
High Priority 57 questions	Frequent practical and design questions. Answer with the approach, the code idea and a real example from your framework.
Medium Priority 28 questions	Occasionally asked, usually as follow-ups or when your resume mentions the topic.
Low Priority / Quick Revision 3 questions	Niche protocols and rare scenarios. Know the one-line idea.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
API & HTTP Fundamentals	Precise answers on HTTP methods, idempotency, status codes and REST vs SOAP.
REST Assured: Requests	Can you write GET/POST/PUT/DELETE requests, request/response specifications and parameters live?
REST Assured: Validation	JsonPath extraction, schema validation, nested JSON, response time, headers: beyond status code.
Authentication & Security	Basic vs OAuth2 vs token flows in code, JWT checks, and simple security tests (SQLi, XSS).
Framework Design	The core 4+ years topic: BaseTest, POJOs, config, utilities, data-driven, chaining, structure.
Postman & Newman	Scripting in the Tests tab, variables, chaining and running collections headless.
CI/CD & Reporting	How API tests run automatically and how failures are triaged and reported.
Scenario-Based	Approach questions: no documentation, pagination, async, versioning, dependencies.
Reliability & Performance	Flaky API tests, retries, timeouts, response-time checks.
Mocking & WireMock	Why and how you mock dependencies when the real service is unavailable.
Contract Testing	Consumer-driven contracts (Pact) vs schema validation.

Note: one entry ('API Testing in Java - Wrote code to validate request and response payloads using REST-assured') was a resume line; it is now part of the JSON validation question. Short fragments such as 'Request chaining' or 'Logging requests and responses' are phrased as full questions and merged with matching ones.

Table of Contents

Section 1: Critical (Must Prepare) (33 questions) . . . 5
API Testing & HTTP Fundamentals (5) . . . 5
REST Assured: Requests, Specifications & Parameters (7) . . . 6
REST Assured: Response Validation & JSON Handling (6) . . . 7
Authentication, Security & Rate Limiting (2) . . . 8
API Automation Framework Design (6) . . . 8
Postman Scripting & Newman (4) . . . 9
CI/CD & Reporting for API Tests (1) . . . 10
Scenario-Based API Testing (1) . . . 10
Reliability, Retry & Performance (1) . . . 11

Section 2: High Priority (57 questions) . . . 12
API Testing & HTTP Fundamentals (5) . . . 12
REST Assured: Requests, Specifications & Parameters (4) . . . 12
REST Assured: Response Validation & JSON Handling (6) . . . 13
Authentication, Security & Rate Limiting (3) . . . 14
API Automation Framework Design (9) . . . 15
Postman Scripting & Newman (9) . . . 16
CI/CD & Reporting for API Tests (3) . . . 18
Scenario-Based API Testing (12) . . . 18
Reliability, Retry & Performance (1) . . . 19
Mocking & WireMock (2) . . . 20
API Contract Testing (3) . . . 20

Section 3: Medium Priority (28 questions) . . . 21
API Testing & HTTP Fundamentals (2) . . . 21
REST Assured: Requests, Specifications & Parameters (3) . . . 21
REST Assured: Response Validation & JSON Handling (3) . . . 21
Authentication, Security & Rate Limiting (1) . . . 22
API Automation Framework Design (2) . . . 22
Postman Scripting & Newman (5) . . . 22
CI/CD & Reporting for API Tests (1) . . . 23
Scenario-Based API Testing (7) . . . 23
Reliability, Retry & Performance (1) . . . 24
API Contract Testing (3) . . . 24

Section 4: Low Priority / Quick Revision (3 questions) . . . 25
Authentication, Security & Rate Limiting (1) . . . 25
Scenario-Based API Testing (2) . . . 25

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
API Testing & HTTP Fundamentals	5 Qs, p.5	5 Qs, p.12	2 Qs, p.21	-	**12**
REST Assured: Requests, Specifications & Parameters	7 Qs, p.6	4 Qs, p.12	3 Qs, p.21	-	**14**
REST Assured: Response Validation & JSON Handling	6 Qs, p.7	6 Qs, p.13	3 Qs, p.21	-	**15**
Authentication, Security & Rate Limiting	2 Qs, p.8	3 Qs, p.14	1 Qs, p.22	1 Qs, p.25	**7**
API Automation Framework Design	6 Qs, p.8	9 Qs, p.15	2 Qs, p.22	-	**17**
Postman Scripting & Newman	4 Qs, p.9	9 Qs, p.16	5 Qs, p.22	-	**18**
CI/CD & Reporting for API Tests	1 Qs, p.10	3 Qs, p.18	1 Qs, p.23	-	**5**
Scenario-Based API Testing	1 Qs, p.10	12 Qs, p.18	7 Qs, p.23	2 Qs, p.25	**22**
Reliability, Retry & Performance	1 Qs, p.11	1 Qs, p.19	1 Qs, p.24	-	**3**
Mocking & WireMock	-	2 Qs, p.20	-	-	**2**
API Contract Testing	-	3 Qs, p.20	3 Qs, p.24	-	**6**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (33 questions)

Asked in almost every API automation interview at 4+ years. Be ready to explain the concept and to write the REST Assured code or Postman script on the spot.

API Testing & HTTP Fundamentals (5)

1. What is API testing and why is it important?

   Answer: API testing checks the business-logic layer directly by sending requests and validating responses (status, data, headers, performance, security) without the UI. It is important because it finds defects early, runs faster, is more stable than UI tests and covers logic the UI may hide.

   Example: Test POST /orders directly to confirm the order is created with correct totals, before the checkout UI is ready.

2. Explain the HTTP methods used in RESTful APIs (GET, POST, PUT, PATCH, DELETE). **[asked x2 in your bank]**

   Answer: GET reads data, POST creates, PUT replaces a resource completely, PATCH updates part of it, DELETE removes it.

   Example: GET /products/101, POST /orders, PUT /users/7, PATCH /orders/1001 (status), DELETE /cart/items/5.

   ```
   GET    /products/101
   POST   /orders           { "productId": 101, "qty": 2 }
   PATCH  /orders/1001      { "status": "CANCELLED" }
   DELETE /cart/items/5
   ```

3. What is idempotency in HTTP methods?

   Answer: An operation is idempotent when repeating it gives the same result as doing it once. GET, PUT, DELETE, HEAD and OPTIONS are idempotent; POST and usually PATCH are not.

   Example: Calling DELETE /cart/items/5 twice leaves the cart the same; calling POST /orders twice creates two orders.

4. Explain HTTP status code categories and what 200, 201, 400, 401, 404 and 500 mean. **[asked x7 in your bank]**

   Answer: 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error. Common: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 429 Too Many Requests, 500, 502, 503.

   Example: POST /orders returns 201; wrong token 401; another user's order 403; unknown order 404; duplicate coupon 409.

5. What is the difference between SOAP and REST APIs? How do you test SOAP? **[asked x2 in your bank]**

   Answer: SOAP is a strict XML protocol with WSDL contract and WS-Security. REST is a lightweight style that supports JSON/XML and HTTP methods. To test SOAP, send an XML envelope with Content-Type text/xml and a SOAPAction header (SoapUI or Postman) and validate against the WSDL/XSD.

   Example: Legacy payment gateway uses SOAP; the order service uses REST.

   ```
   POST /PaymentService
   Content-Type: text/xml; charset=utf-8
   SOAPAction: "ProcessPayment"

   <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
     <soapenv:Body><ProcessPayment><amount>999</amount></ProcessPayment></soapenv:Body>
   </soapenv:Envelope>
   ```

REST Assured: Requests, Specifications & Parameters (7)

6. API automation: RestAssured vs Postman tests. When do you use which?

> **Answer:** Postman is great for quick exploratory/manual testing and simple automated checks. RestAssured (Java) suits large automated suites: code reuse, complex logic, version control, TestNG/CI integration and shared utilities with Selenium.
>
> **Example:** Postman to explore a new payment endpoint; RestAssured regression suite running in Jenkins.

```
given().header("Authorization", "Bearer " + token)
  .when().get("/orders/1001")
  .then().statusCode(200).body("status", equalTo("PLACED"));
```

7. What is REST Assured? Explain its structure and components, and how you set it up in a project. **[asked x6 in your bank]**

> **Answer:** REST Assured is a Java library for testing REST APIs with a given-when-then style. given() sets up the request (base URI, headers, params, body, auth), when() performs the call, then() validates the response. Setup: add rest-assured, TestNG/JUnit, json-schema-validator and Jackson as Maven dependencies.
>
> **Example:** A Maven test project with rest-assured and testng calling the shop's /products and /orders APIs.

```
<dependency>
  <groupId>io.rest-assured</groupId><artifactId>rest-assured</artifactId>
  <version>5.4.0</version><scope>test</scope>
</dependency>
<!-- also: json-schema-validator, testng (or junit), jackson-databind -->
```

8. Write a basic GET request using REST Assured. **[asked x4 in your bank]**

> **Answer:** Use given().when().get(path).then() with a status check and body validation.
>
> **Example:** GET a product by id and check its name.

```
given().baseUri("https://api.shop.com")
  .when().get("/products/101")
  .then().statusCode(200).body("name", equalTo("Phone"));
```

9. How do you send a POST request with a JSON body using REST Assured? **[asked x3 in your bank]**

> **Answer:** Set the content type to JSON and pass the body as a JSON string, Map or POJO; validate 201 and the returned data.
>
> **Example:** Create an order and check the response.

```
given().contentType(ContentType.JSON)
  .body("{\"productId\":101,\"qty\":2}")
  .when().post("/orders")
  .then().statusCode(201).body("status", equalTo("PLACED"));
```

10. What are Request Specification and Response Specification? How do you create and reuse them (base URI)? **[asked x9 in your bank]**

> **Answer:** RequestSpecification stores reusable request settings (base URI, headers, auth, content type); ResponseSpecification stores common expectations (status, content type, time). Build with RequestSpecBuilder/ResponseSpecBuilder and pass with spec(), or set globally with RestAssured.requestSpecification.
>
> **Example:** Every shop API test reuses the base URL, JSON content type and auth token.

```
RequestSpecification req = new RequestSpecBuilder()
    .setBaseUri("https://api.shop.com").setContentType(ContentType.JSON)
    .addHeader("Authorization", "Bearer " + token).build();
ResponseSpecification res = new ResponseSpecBuilder()
    .expectStatusCode(200).expectContentType(ContentType.JSON).build();
given().spec(req).when().get("/orders/1001").then().spec(res);
```

11. How do you handle query parameters and path parameters in REST Assured? **[asked x5 in your bank]**

> **Answer:** Use queryParam() for filters (?category=shoes) and pathParam() with {placeholders} for resource ids.
>
> **Example:** GET /products?category=shoes&page=2 and GET /orders/1001.

```
given().queryParam("category", "shoes").queryParam("page", 2)
  .when().get("/products").then().statusCode(200);
given().pathParam("id", 1001)
  .when().get("/orders/{id}").then().statusCode(200);
```

12. How do you log request and response details (to console or a file)? **[asked x6 in your bank]**

> **Answer:** Use log().all() or log().ifValidationFails() on the request and response; to write to a file use the RequestLoggingFilter/ResponseLoggingFilter with a PrintStream.
>
> **Example:** Save request and response of failed order tests to a log file.

```
PrintStream ps = new PrintStream(new FileOutputStream("target/api.log"));
given().filter(RequestLoggingFilter.logRequestTo(ps))
  .filter(ResponseLoggingFilter.logResponseTo(ps))
  .when().get("/orders/1001").then().log().ifValidationFails();
```

REST Assured: Response Validation & JSON Handling (6)

13. How do you validate an API response? What are the most common validations? **[asked x2 in your bank]**

> **Answer:** Validate status code, response body values and types, headers, response time and the schema; also check error responses. Common: status, key fields, schema and time.
>
> **Example:** Order API: 201, status PLACED, Content-Type JSON, total correct, under 1 second.

```
given().spec(req).body(orderJson).when().post("/orders")
  .then().statusCode(201).contentType(ContentType.JSON)
  .body("status", equalTo("PLACED")).time(lessThan(1000L));
```

14. How do you validate the status code and response body with assertions (and multiple assertions in one request)? **[asked x7 in your bank]**

> **Answer:** Chain several body() checks (or one body() with multiple pairs) in the same then() block; use Hamcrest matchers. TestNG SoftAssert helps when you want all failures reported.
>
> **Example:** Validate id, status and item count of an order in one request.

```
given().when().get("/orders/1001").then().statusCode(200)
  .body("id", equalTo(1001), "status", equalTo("PLACED"), "items.size()", greaterThan(0));
```

15. How do you validate JSON response data, extract values and use JsonPath (including checking a key is present)? **[asked x10 in your bank]**

> **Answer:** Use response.jsonPath() (or path()) to read values with a path expression, then assert in Java. To check a key exists use notNullValue() or hasKey().
>
> **Example:** Extract the order id and the first item SKU.

```
Response res = given().when().get("/orders/1001");
JsonPath js = res.jsonPath();
int id = js.getInt("id");
String sku = js.getString("items[0].sku");
boolean hasCoupon = js.getMap("$").containsKey("couponCode");
```

16. How do you validate JSON schema in REST Assured? What is schema validation and why is it important? **[asked x7 in your bank]**

> **Answer:** Add the json-schema-validator dependency and use matchesJsonSchemaInClasspath with a schema file; it fails when fields, types or required keys change, so it catches contract changes.
>
> **Example:** product-schema.json checks id is an integer and price a number.

```
given().when().get("/products/101").then().statusCode(200)
  .body(matchesJsonSchemaInClasspath("schemas/product-schema.json"));
```

17. How do you validate response time? **[asked x4 in your bank]**

> **Answer:** Use the time() matcher with lessThan, or read response.getTime() and assert it.
>
> **Example:** Search API must respond within 800 ms.

```
given().when().get("/products?query=phone")
  .then().time(lessThan(800L), TimeUnit.MILLISECONDS);
long ms = given().get("/products").getTime();
```

18. How do you test error handling, error responses, error codes and negative scenarios? **[asked x6 in your bank]**

> **Answer:** Send invalid, missing and unauthorized data and assert the status code and error body (and that no stack trace is exposed).
>
> **Example:** Quantity -1 must return 400 with a clear message.

```
given().contentType(ContentType.JSON).body("{\"productId\":101,\"qty\":-1}")
  .when().post("/orders")
  .then().statusCode(400).body("error", containsString("qty"));
```

Authentication, Security & Rate Limiting (2)

19. How do you handle authentication in REST Assured (basic, OAuth2, token-based, dynamic tokens)? **[asked x9 in your bank]**

> **Answer:** Basic: auth().basic(). Token/OAuth2: auth().oauth2(token) or an Authorization: Bearer header. For dynamic tokens call the login API once (for example in @BeforeClass), extract the token and reuse it.
>
> **Example:** Login as a customer, store the token and use it for cart and order calls.

```
String token = given().contentType(ContentType.JSON).body(creds)
    .when().post("/auth/login").then().extract().path("token");
given().auth().oauth2(token).when().get("/orders").then().statusCode(200);
given().auth().preemptive().basic("user", "pass");
```

20. What is the difference between authentication and authorization?

> **Answer:** Authentication proves who you are (401 if it fails). Authorization decides what you are allowed to do (403 if not permitted).
>
> **Example:** A valid customer token can view own orders (200) but gets 403 on the admin /users endpoint.

API Automation Framework Design (6)

21. How do you structure an API automation framework?

> **Answer:** Use a layered structure: base (BaseTest, specs), config, endpoints/API services, pojo models, tests, utilities (validators, data readers), test data and schemas in resources, reports and CI files.
>
> **Example:** src/main: endpoints, pojos, utils; src/test: tests per module; resources: schemas, test data, config.

22. Implement a BaseTest class for API tests.

> **Answer:** BaseTest sets common setup once: base URI, auth, request specification, logging and cleanup; test classes extend it.
>
> **Example:** OrderApiTest extends BaseTest and reuses the spec and token.

```
public class BaseTest {
  protected RequestSpecification req;
  @BeforeClass public void setup() {
    RestAssured.baseURI = ConfigReader.get("base.url");
    req = new RequestSpecBuilder().setContentType(ContentType.JSON)
        .addHeader("Authorization", "Bearer " + AuthHelper.token()).build();
  }
}
```

23. Write a POJO class for serialization/deserialization. How do you use Jackson/Gson and POJOs in API tests? **[asked x3 in your bank]**

> **Answer:** A POJO is a plain class with private fields, getters/setters and a default constructor. Jackson/Gson serialises it to JSON for the request and deserialises the response into it.
>
> **Example:** An Order POJO built in the test and sent as the body of POST /orders.

```
public class Order {
  private int productId; private int qty;
  public Order() { }
  public Order(int productId, int qty) { this.productId = productId; this.qty = qty; }
  public int getProductId() { return productId; }
  public int getQty() { return qty; }
}
given().contentType(ContentType.JSON).body(new Order(101, 2)).post("/orders");
Order created = given().get("/orders/1001").as(Order.class);
```

24. How do you implement data-driven testing for APIs (TestNG DataProvider, JUnit, Excel/JSON)? **[asked x7 in your bank]**

> **Answer:** Use a TestNG DataProvider (or JUnit parameterized tests) that reads rows from Excel/JSON/CSV and passes them to the test, so one test covers many inputs.
>
> **Example:** Create orders with different quantities and coupon codes from a JSON file.

```
@DataProvider(name = "orders")
public Object[][] orders() {
  return new Object[][] {{101, 1, 201}, {101, 0, 400}, {999, 1, 404}};
}
@Test(dataProvider = "orders")
public void createOrder(int product, int qty, int expected) {
  given().spec(req).body(new Order(product, qty)).post("/orders").then().statusCode(expected);
}
```

25. How do you integrate REST Assured with TestNG or JUnit? **[asked x2 in your bank]**

> **Answer:** Write REST Assured calls inside @Test methods, use @BeforeClass for setup, run with testng.xml or Maven Surefire; REST Assured works the same with JUnit.
>
> **Example:** mvn test runs OrderApiTest and ProductApiTest from testng.xml.

```
@Test(groups = "smoke")
public void getProduct() {
  given().spec(req).get("/products/101").then().statusCode(200);
}
```

26. How do you implement request chaining (extract a value and pass it to the next request)? **[asked x2 in your bank]**

> **Answer:** Extract a value from one response and use it in the next request in the same test.
>
> **Example:** Create an order, capture its id, then fetch and cancel it.

```
int orderId = given().spec(req).body(order).post("/orders")
    .then().statusCode(201).extract().path("id");
given().spec(req).pathParam("id", orderId).get("/orders/{id}").then().statusCode(200);
```

Postman Scripting & Newman (4)

27. What are Collections, Environments and Variables in Postman? How do you use environment variables in scripts and import/export them? **[asked x5 in your bank]**

> **Answer:** A Collection groups requests; an Environment holds values per stage (QA/UAT); variables (global, collection, environment, data, local) are used as {{name}} in requests and via pm.* in scripts. Export/import collections and environments as JSON.
>
> **Example:** QA and UAT environments hold different baseUrl and token values.

```
pm.environment.set("baseUrl", "https://qa.shop.com");
const url = pm.environment.get("baseUrl");
```

28. What is the difference between Pre-request Script and Tests in Postman?

> **Answer:** Pre-request Script runs before the request is sent (set variables, build data, create tokens or signatures). Tests run after the response arrives (assertions on status, body, headers, time).
>
> **Example:** Pre-request generates a random email; Tests verify 201 and the returned id.

```
// Pre-request
pm.variables.set("email", "user" + Date.now() + "@test.com");
// Tests
pm.test("Created", () => pm.response.to.have.status(201));
```

29. How do you write a basic test script in Postman (status code, response time, JSON body assertions)? Which scripting language does it use? **[asked x4 in your bank]**

> **Answer:** Write assertions in the Tests tab using pm.test and pm.expect. The scripting language is JavaScript (Postman sandbox).
>
> **Example:** Check status, response time and the order id.

```
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.test("Fast", () => pm.expect(pm.response.responseTime).to.be.below(800));
pm.test("Has id", () => pm.expect(pm.response.json().id).to.exist);
```

30. How do you chain requests in Postman (dynamic tokens, store and reuse values, capture header values, call one request from another, sequencing)? **[asked x7 in your bank]**

> **Answer:** Store values from a response in variables and reuse them as {{var}} in later requests; read headers with pm.response.headers; call another request with pm.sendRequest; control order with the collection sequence or setNextRequest.
>
> **Example:** Login stores the token, create-order stores orderId, payment uses both.

```
pm.environment.set("token", pm.response.json().token);
pm.collectionVariables.set("etag", pm.response.headers.get("ETag"));
pm.sendRequest({ url: pm.environment.get("baseUrl") + "/health", method: "GET" },
  (err, res) => console.log(res.code));
```

CI/CD & Reporting for API Tests (1)

31. How do you run API tests in CI/CD pipelines (Newman or REST Assured with Jenkins/Maven)? How do you trigger them after deployment? **[asked x6 in your bank]**

> **Answer:** Run mvn test (REST Assured) or newman run in a Jenkins/GitHub Actions stage, pass the environment as a parameter, publish the report and trigger the job after deployment through a pipeline stage or webhook.
>
> **Example:** After the QA deployment, a downstream job runs the API regression and posts results.

```
pipeline {
  agent any
  stages {
    stage('API Tests') { steps { sh 'mvn clean test -Denv=qa' } }
  }
  post { always { junit 'target/surefire-reports/*.xml' } }
}
```

Scenario-Based API Testing (1)

32. How do you approach testing a new API when no documentation is available, or the requirements are incomplete and the behaviour changes often? **[asked x2 in your bank]**

> **Answer:** Explore the app to discover the API (browser DevTools, Swagger/OpenAPI, proxy tools), ask developers/BAs, build a Postman collection from what you find, test happy and negative paths, record assumptions and confirm behaviour when it changes.
>
> **Example:** Captured the cart API calls from the browser Network tab and turned them into a Postman collection.

Reliability, Retry & Performance (1)

33. How do you handle flaky or intermittent API tests? Implement retry logic for failed requests. **[asked x4 in your bank]**

> **Answer:** Find the cause first (timing, shared data, unstable environment, async jobs). Fix with proper waits/polling (Awaitility), unique data and isolated tests; retry only on infrastructure errors such as 5xx, using IRetryAnalyzer or a limited retry in the client.
>
> **Example:** A job-status API is polled until COMPLETED instead of failing after one call.

```
await().atMost(30, TimeUnit.SECONDS).pollInterval(2, TimeUnit.SECONDS)
  .until(() -> given().get("/jobs/55").path("status").equals("COMPLETED"));
```

Section 2: High Priority (57 questions)

Frequent practical and design questions. Answer with the approach, the code idea and a real example from your framework.

API Testing & HTTP Fundamentals (5)

34. What is the difference between API testing and UI testing frameworks?

> **Answer:** API test frameworks send requests directly to the service (no browser), so they are faster, more stable and good for business logic and data checks. UI frameworks (Selenium) test the user interface in a browser and are slower and more fragile.
>
> **Example:** Check order creation through the API in seconds; use the UI test only for the checkout page layout.

35. What is the difference between URI, URL and endpoint?

> **Answer:** URI identifies a resource; a URL is a URI that also tells where to find it (protocol and address); an endpoint is the specific URL where the API accepts requests.
>
> **Example:** Endpoint: https://api.shop.com/v1/orders (a URL); resource URI: /orders/1001.

36. What are headers in API requests and responses?

> **Answer:** Headers carry metadata: Content-Type, Accept, Authorization, Cache-Control, cookies in requests and status-related and caching information in responses.
>
> **Example:** Authorization: Bearer <token> in the request; Content-Type: application/json in the response.

37. What is JSON and why is it preferred over XML?

> **Answer:** JSON is lightweight, readable and easy to parse with native types and arrays, so payloads are smaller and faster. XML is more verbose but supports namespaces, attributes and strong schemas.
>
> **Example:** The shop's mobile API uses JSON; the legacy payment service uses XML.

38. What is the difference between smoke, regression and load testing in APIs?

> **Answer:** Smoke: quick check that key endpoints work after a deployment. Regression: re-run existing tests to confirm changes did not break anything. Load: many concurrent requests to measure response time and throughput.
>
> **Example:** Smoke: /health, login, /products. Regression: full order collection. Load: 500 users hitting search.

REST Assured: Requests, Specifications & Parameters (4)

39. How do you perform PUT (with validation) and DELETE requests in REST Assured? Write sample code. **[asked x3 in your bank]**

> **Answer:** PUT sends the full updated resource and validates the response; DELETE removes the resource and returns 204/200; confirm with a follow-up GET returning 404.
>
> **Example:** Update a product's price with PUT, delete a cart item with DELETE.

```
given().spec(req).body(product).when().put("/products/101")
  .then().statusCode(200).body("price", equalTo(899));
given().spec(req).when().delete("/cart/items/5").then().statusCode(204);
```

40. How do you set headers and cookies, and extract cookies from a response? **[asked x4 in your bank]**

> **Answer:** Use header()/headers() and cookie()/cookies() on the request; read cookies with response.getCookie() or detailedCookies().
>
> **Example:** Send a session cookie and read the cookie set at login.

```
Response res = given().header("Accept", "application/json").cookie("lang", "en")
    .when().post("/login");
String session = res.getCookie("SESSIONID");
```

41. How do you test APIs with different content types (JSON, XML, form data)? **[asked x2 in your bank]**

> **Answer:** Set contentType(ContentType.JSON/XML/URLENC) and send the matching body; use formParam() for form data.
>
> **Example:** Login by form data, create orders with JSON and legacy orders with XML.

```
given().contentType(ContentType.URLENC).formParam("user", "asha").formParam("pass", "x")
  .post("/login");
given().contentType(ContentType.XML).body("<order><sku>101</sku></order>").post("/orders");
```

42. How do you perform file upload and multipart form-data requests? **[asked x4 in your bank]**

> **Answer:** Use multiPart() with a file (and extra form fields); REST Assured sets multipart/form-data automatically. Verify 201 and file details.
>
> **Example:** Upload a product image.

```
given().multiPart("file", new File("data/product.png"), "image/png")
  .multiPart("productId", "101")
  .when().post("/products/101/image").then().statusCode(201);
```

REST Assured: Response Validation & JSON Handling (6)

43. How do you validate response headers, cookies and content type? **[asked x3 in your bank]**

> **Answer:** Validate headers with header()/headers(), cookies with cookie() and the content type with contentType().
>
> **Example:** Response must be JSON and must set a session cookie.

```
given().when().get("/orders/1001").then()
  .header("Cache-Control", containsString("no-cache"))
  .contentType(ContentType.JSON)
  .cookie("SESSIONID", notNullValue());
```

44. How do you validate JSON arrays and lists containing specific values? **[asked x2 in your bank]**

> **Answer:** Use Hamcrest collection matchers: hasItems, hasSize, containsInAnyOrder, everyItem, or extract the list and assert in Java.
>
> **Example:** Product list must contain Phone and Case and have 3 items.

```
given().get("/products").then()
  .body("items.name", hasItems("Phone", "Case"))
  .body("items", hasSize(3));
```

45. How do you handle nested and deeply nested JSON responses? **[asked x4 in your bank]**

> **Answer:** Use the dot/index path for nested values, Groovy closures like findAll for filtering, or map the response to POJOs for deep structures.
>
> **Example:** Read the SKU of the second item of the first order.

```
given().get("/customers/7").then()
  .body("orders[0].items[1].sku", equalTo("S2"))
  .body("orders.items.flatten().findAll { it.price > 500 }.sku", hasItem("S9"));
```

46. How do you handle dynamic data and dynamic payloads or JSON files? **[asked x2 in your bank]**

> **Answer:** Build payloads at runtime from a template file or Map/POJO with placeholders, fill unique values (UUID, timestamp, Faker) and avoid hard-coded data.
>
> **Example:** A new customer with a unique email per run.

```
String body = new String(Files.readAllBytes(Paths.get("data/customer.json")))
    .replace("{{email}}", "user" + System.currentTimeMillis() + "@test.com");
given().spec(req).body(body).post("/customers");
```

47. How do you compare two API response lists for equality ignoring order?

Answer: Extract both lists and compare with containsInAnyOrder, or sort both lists, or use TestNG assertEqualsNoOrder.

Example: Product ids returned by two environments must match regardless of order.

```
List<Integer> a = res1.jsonPath().getList("items.id");
List<Integer> b = res2.jsonPath().getList("items.id");
assertThat(a, containsInAnyOrder(b.toArray()));
```

48. What are soft and hard assertions?

Answer: A hard assertion stops the test at the first failure; a soft assertion (TestNG SoftAssert/AssertJ SoftAssertions) records failures and reports them together with assertAll().

Example: Check name, price and stock of one product and see all mismatches at once.

```
SoftAssert sa = new SoftAssert();
sa.assertEquals(res.path("name"), "Phone");
sa.assertEquals((int) res.path("stock"), 10);
sa.assertAll();
```

Authentication, Security & Rate Limiting (3)

49. How do you test API authentication (OAuth, JWT)? How do you validate JWT tokens? **[asked x2 in your bank]**

Answer: Test valid token (200), missing, expired, tampered or wrong-role tokens (401/403). To validate a JWT decode its payload, check claims such as exp, role and issuer, and confirm the signature is rejected when altered.

Example: An expired token on GET /orders must return 401.

```
String[] parts = token.split("\\.");
String payload = new String(Base64.getUrlDecoder().decode(parts[1]));
long exp = new ObjectMapper().readTree(payload).get("exp").asLong();
Assert.assertTrue(exp > Instant.now().getEpochSecond());
```

50. What is API security testing? How do you test broken authentication, SQL injection and XSS on an API? **[asked x6 in your bank]**

Answer: Security testing covers broken authentication, injection (SQLi/XSS), IDOR, excessive data exposure and weak rate limits. Send malicious payloads and check they are rejected or safely handled with no error or data leak (on an authorised test environment).

Example: Searching for ' OR '1'='1 must not return all products or a SQL error.

```
given().queryParam("q", "' OR '1'='1")
  .when().get("/products")
  .then().statusCode(anyOf(is(200), is(400)))
  .body(not(containsString("SQL syntax")));
```

51. How do you test API rate limiting (throttling) and handle it in automation? **[asked x4 in your bank]**

Answer: Send requests in a loop beyond the limit and expect 429 with Retry-After; in the framework handle 429 by waiting and retrying instead of failing.

Example: More than 100 search requests per minute return 429.

```
int limited = 0;
for (int i = 0; i < 120; i++)
  if (given().get("/products").statusCode() == 429) limited++;
Assert.assertTrue(limited > 0);
```

API Automation Framework Design (9)

52. Design a config.properties management class and manage environment-based execution (dev, staging, prod). **[asked x4 in your bank]**

> **Answer:** Load environment-specific properties (dev, staging, prod) based on a system property such as -Denv, and expose them through a static getter.
>
> **Example:** -Denv=qa loads qa.properties with the QA base URL.

```
public class ConfigReader {
  private static final Properties p = new Properties();
  static {
    String env = System.getProperty("env", "qa");
    try (InputStream in = ConfigReader.class.getResourceAsStream("/" + env + ".properties")) {
      p.load(in);
    } catch (IOException e) { throw new RuntimeException(e); }
  }
  public static String get(String key) { return p.getProperty(key); }
}
```

53. Implement the Singleton pattern for an API client.

> **Answer:** A Singleton API client keeps one shared instance (private constructor, static getInstance) that holds the base spec; use a thread-safe holder.
>
> **Example:** One ApiClient shared by all tests with a common base spec.

```
public class ApiClient {
  private static class Holder { static final ApiClient INSTANCE = new ApiClient(); }
  private final RequestSpecification spec = new RequestSpecBuilder()
      .setBaseUri(ConfigReader.get("base.url")).build();
  private ApiClient() { }
  public static ApiClient getInstance() { return Holder.INSTANCE; }
  public RequestSpecification spec() { return spec; }
}
```

54. Create a response validator utility and a reusable API helper class. **[asked x2 in your bank]**

> **Answer:** A helper class wraps common calls (get, post) and a validator class holds reusable checks, so tests stay short and consistent.
>
> **Example:** ApiHelper.post("/orders", body) and ResponseValidator.assertStatus(res, 201).

```
public class ApiHelper {
  public static Response post(String path, Object body) {
    return given().spec(ApiClient.getInstance().spec()).body(body).post(path);
  }
}
public class ResponseValidator {
  public static void assertStatus(Response r, int code) {
    Assert.assertEquals(r.statusCode(), code);
  }
}
```

55. Design endpoint management using Enums. Design a page object model for API endpoints. **[asked x2 in your bank]**

> **Answer:** Keep endpoints in an enum (one place to change) and create an API service class per resource that wraps its calls, similar to a page object for APIs.
>
> **Example:** OrderApi.create(order) uses Endpoint.ORDERS.

```
public enum Endpoint {
  PRODUCTS("/products"), ORDERS("/orders"), CART("/cart");
  private final String path;
  Endpoint(String path) { this.path = path; }
  public String path() { return path; }
}
public class OrderApi {
  public Response create(Order o) { return ApiHelper.post(Endpoint.ORDERS.path(), o); }
}
```

56. Create an end-to-end API test scenario.

> **Answer:** Chain real calls: create user, login, add to cart, place order, pay and verify the final order status.
>
> **Example:** Register, login, add item, order and confirm status PAID.

```
String token = login(user);
given().auth().oauth2(token).body(item).post("/cart/items").then().statusCode(201);
int orderId = given().auth().oauth2(token).post("/orders").then().extract().path("id");
given().auth().oauth2(token).get("/orders/" + orderId)
  .then().body("status", equalTo("PLACED"));
```

57. How do you manage test data?

> **Answer:** Use JSON/Excel files, builders and Faker for data, create and clean data through APIs or the DB, keep data per environment and make it unique for parallel runs.
>
> **Example:** Each test creates its own customer by API and deletes it afterwards.

58. Explain TestNG groups for API test organization. How do you run tests in parallel? **[asked x2 in your bank]**

> **Answer:** Tag tests with TestNG groups (smoke, regression) and select them in testng.xml; run in parallel with parallel and thread-count. REST Assured is thread-safe if you avoid changing static global settings during the run.
>
> **Example:** Run smoke group on every commit and regression nightly in parallel.

```
@Test(groups = {"smoke", "orders"}) public void createOrder() { }
// testng.xml: <suite parallel="methods" thread-count="4">
```

59. How do you integrate API and UI tests?

> **Answer:** Use APIs to prepare data and to log in quickly for UI tests, and the API to verify backend results after a UI action; keep one framework or shared utilities.
>
> **Example:** Create the cart by API and open the checkout page directly in Selenium.

```
String sessionId = given().body(creds).post("/login").getCookie("SESSIONID");
driver.manage().addCookie(new Cookie("SESSIONID", sessionId));
```

60. What are best practices for API automation? How do you keep API tests maintainable? **[asked x5 in your bank]**

> **Answer:** Keep tests independent, use specs and helpers, avoid hard-coded data, validate schema and negative cases, use clear names, run in CI and keep reports and logs.
>
> **Example:** One helper builds every order request, so a change is made in one place.

Postman Scripting & Newman (9)

61. What is Newman? How do you run Postman tests headlessly and generate HTML reports? **[asked x4 in your bank]**

> **Answer:** Export the collection and environment and run with Newman (npm install -g newman). In CI (Jenkins/GitHub Actions) run the same command and publish the report.
>
> **Example:** Nightly Jenkins job runs the shop collection against QA.

```
newman run shop.postman_collection.json -e qa.postman_environment.json \
  -d users.csv -r cli,htmlextra
```

62. How do you send authentication credentials in Postman?

> **Answer:** Use the Authorization tab (Basic, Bearer, OAuth 2.0), set it at collection level for all requests, or add an Authorization header using variables such as {{token}}.
>
> **Example:** Collection-level Bearer {{token}} for all cart and order requests.

```
Authorization: Bearer {{token}}
```

63. What is the Collection Runner?

> **Answer:** The Collection Runner runs the requests of a collection in order with iterations, data files, delay and shows pass/fail; Newman is the command-line version.
>
> **Example:** Run the whole order collection for 10 iterations with a CSV file.

64. How do you run data-driven tests and parameterize requests in Postman? **[asked x2 in your bank]**

> **Answer:** Use the Collection Runner with a CSV or JSON data file; columns become variables used in the request as {{username}} or in scripts with pm.iterationData. Each row is one iteration.
>
> **Example:** Run the login request with many valid and invalid user rows from a CSV file.

```
// users.csv: username,password,expectedStatus
const expected = Number(pm.iterationData.get("expectedStatus"));
pm.test("Status matches", () => pm.response.to.have.status(expected));
```

65. How do you validate JSON schema and arrays in Postman? **[asked x3 in your bank]**

> **Answer:** Use pm.response.to.have.jsonSchema(schema) for schema checks and array assertions with pm.expect for lists.
>
> **Example:** Check every product in the list has id and price.

```
const schema = { type: "array", items: { type: "object", required: ["id", "price"] } };
pm.test("Schema", () => pm.response.to.have.jsonSchema(schema));
pm.test("Not empty", () => pm.expect(pm.response.json()).to.be.an("array").that.is.not.empty);
```

66. How do you run only selected tests, folders or requests from a collection? **[asked x2 in your bank]**

> **Answer:** In the Runner select folders/requests; with Newman use --folder to run specific folders or requests.
>
> **Example:** Run only the Cart and Orders folders after a cart change.

```
newman run shop.json --folder "Cart" --folder "Orders"
```

67. How do you handle loops, conditional logic, random data and console logging in Postman scripts? **[asked x4 in your bank]**

> **Answer:** Use JavaScript in scripts: forEach loops, if/else on variables, random values and console.log for debugging.
>
> **Example:** Log each SKU and pick a random quantity.

```
pm.response.json().items.forEach(i => console.log(i.sku));
const qty = Math.floor(Math.random() * 5) + 1;
if (pm.environment.get("env") === "qa") pm.variables.set("qty", qty);
```

68. How do you retry failed requests in Postman scripts?

> **Answer:** Keep a retry counter in a collection variable and re-run the request with setNextRequest until the limit; always reset the counter afterwards.
>
> **Example:** Retry the payment status request up to 3 times.

```
let tries = Number(pm.collectionVariables.get("tries") || 0);
if (pm.response.code !== 200 && tries < 3) {
  pm.collectionVariables.set("tries", tries + 1);
  postman.setNextRequest(pm.info.requestName);
} else pm.collectionVariables.unset("tries");
```

69. How do you pass query parameters in Postman?

> **Answer:** Add them in the Params tab or the URL (?page=2), or add them in a script with pm.request.url.query.add.
>
> **Example:** GET {{baseUrl}}/products?category=shoes&page=2.

```
pm.request.url.query.add({ key: "page", value: "2" });
```

CI/CD & Reporting for API Tests (3)

70. If an API test fails in CI/CD, how do you troubleshoot?

> **Answer:** Read the CI console log and failed assertion, reproduce locally, check environment/URL, data and token expiry, service health and recent changes, then decide whether it is a defect, flaky test or environment issue and rerun.
>
> **Example:** A 401 in CI after a token secret rotated: update the credential, not the test.

71. How do you pass dynamic environment variables in CI/CD?

> **Answer:** Pass values as system properties or Newman flags and keep secrets in CI credentials.
>
> **Example:** Run the same job against QA or UAT by changing one parameter.

```
mvn test -Denv=uat -Dbase.url=$BASE_URL
newman run shop.json --env-var "baseUrl=$BASE_URL" --env-var "token=$TOKEN"
```

72. How do you publish API test results to a dashboard?

> **Answer:** Publish Allure/Extent or Newman HTML reports in Jenkins (HTML Publisher), or send results to a dashboard such as ReportPortal or Grafana.
>
> **Example:** Jenkins shows the API regression trend and failing tests after every nightly run.

```
publishHTML(target: [reportDir: 'target/allure-report', reportFiles: 'index.html',
  reportName: 'API Report'])
```

Scenario-Based API Testing (12)

73. How do you test API pagination? **[asked x4 in your bank]**

> **Answer:** Test page/size (or limit/offset or cursor) parameters: first/middle/last page, beyond the last page, invalid values, total count, no duplicates or gaps across pages and default page size.
>
> **Example:** GET /products?page=2&size=20 returns 20 items, correct totals and different items than page 1.

```
GET /products?page=1&size=20     -> 20 items, totalPages = N
GET /products?page=999&size=20   -> empty list (or 404)
GET /products?page=0&size=-5     -> 400
```

74. How do you test sorting and filtering in APIs?

> **Answer:** Verify sort order for ascending/descending and for each field, filter combinations, no-match filters, invalid fields and that sorting is consistent across pages.
>
> **Example:** GET /products?category=shoes&sort=price,asc returns only shoes in rising price.

```
const prices = pm.response.json().items.map(i => i.price);
pm.expect(prices).to.eql([...prices].sort((a, b) => a - b));
```

75. How do you test APIs that return asynchronous responses? **[asked x2 in your bank]**

> **Answer:** Expect 202 Accepted with a job id or status URL, poll the status endpoint until completed (with a limit), then verify the final result; also test failure, timeout and callbacks/webhooks.
>
> **Example:** Report export returns 202 and a statusUrl; poll until status is COMPLETED, then download.

76. How do you test APIs that depend on other services, and in a microservices architecture? **[asked x2 in your bank]**

> **Answer:** Test each service alone, then integration; use mocks/stubs for unavailable dependencies, verify data flow and contract between services and test failure handling between them.
>
> **Example:** Order service tested with a mocked payment service and then end to end.

77. How do you test APIs with third-party integrations?

> **Answer:** Use the provider's sandbox/test keys, mock the third party for failures/timeouts, validate request/response mapping, error handling, rate limits and fallback behaviour.
>
> **Example:** Payment gateway sandbox with a test card for success and decline scenarios.

78. How do you validate API caching behaviour? **[asked x2 in your bank]**

> **Answer:** Check Cache-Control, ETag and Last-Modified headers, send If-None-Match and expect 304, and verify data refreshes after an update or when the TTL expires.
>
> **Example:** GET /products/101 returns an ETag; after a price change the new ETag and price are returned.

```
GET /products/101
If-None-Match: "abc123"      -> 304 Not Modified (unchanged)
```

79. How do you test APIs under failure scenarios (including simulated network failures)? **[asked x2 in your bank]**

> **Answer:** Simulate downstream failures, timeouts, DB down and bad input; expect graceful 5xx/503 with a clear message, retries/circuit breaker, and no data corruption or partial records.
>
> **Example:** Stock service down: order API returns 503 and no order is saved.

80. How do you test API concurrency issues?

> **Answer:** Send simultaneous requests (JMeter, Newman parallel runs, RestAssured with threads) to the same resource and check data stays correct: no overselling, no duplicate orders, proper locking or 409 conflicts.
>
> **Example:** Two users buy the last item at once: one succeeds, the other gets 409/out of stock.

81. How do you test APIs that behave differently for different users?

> **Answer:** Get tokens for each role and test allowed and forbidden calls (200 vs 403), data scoping (users see only their own data) and role-based fields.
>
> **Example:** Customer cannot call DELETE /products; admin can.

82. How do you handle API versioning in tests and ensure backward compatibility? **[asked x4 in your bank]**

> **Answer:** Keep the version in config/path and run the same tests per version; confirm old versions still behave the same, new fields are optional and removed behaviour is only in new versions.
>
> **Example:** Run the order tests against /v1 and /v2 and compare.

```
String v = System.getProperty("api.version", "v1");
given().spec(req).when().get("/" + v + "/orders/1001").then().statusCode(200);
```

83. How do you validate data integrity in API testing?

> **Answer:** Check that data sent is stored and returned correctly: compare the POST body with the GET response and the database, check types, relationships and no data loss or duplicates.
>
> **Example:** After POST /orders, confirm order amount in the API, the DB and the orders list are the same.

84. How do you test APIs with file uploads in Postman?

> **Answer:** In Postman choose Body > form-data, set the key type to File and select the file. Verify success (201), stored file details, size/type validation, empty, large and invalid files.
>
> **Example:** Upload a product image of 2 MB: expect 201; upload an .exe: expect 400/415.

```
POST /products/101/image
Content-Type: multipart/form-data
file: product.png
```

Reliability, Retry & Performance (1)

85. How do you handle timeouts, retries and error scenarios, and set a custom request timeout? **[asked x3 in your bank]**

> **Answer:** Set connection and socket timeouts in the REST Assured config, assert on slow responses and retry only safe/idempotent requests with a limit.
>
> **Example:** Fail fast if the payment API takes longer than 10 seconds.

```
RestAssured.config = RestAssuredConfig.config().httpClient(
  HttpClientConfig.httpClientConfig()
    .setParam("http.connection.timeout", 5000).setParam("http.socket.timeout", 10000));
```

Mocking & WireMock (2)

86. What is the purpose and what are the benefits of mocking in API testing? **[asked x2 in your bank]**

> **Answer:** Mocking replaces a real dependency with a controlled fake. Benefits: tests run when the service is unavailable, are faster and stable, can simulate failures/slow replies and allow parallel development.
>
> **Example:** Mock the payment gateway to test a declined payment without a real card.

87. How do you mock an API with WireMock (set up and start the server, configure a response, call it with REST Assured, stop it)? **[asked x7 in your bank]**

> **Answer:** Start a WireMock server, define a stub (request match and response), call it with REST Assured on the mock port and stop the server after the test.
>
> **Example:** Mock GET /products/101 returning a fixed product.

```
WireMockServer wm = new WireMockServer(options().port(8089));
wm.start();
wm.stubFor(get(urlEqualTo("/products/101")).willReturn(aResponse()
    .withStatus(200).withHeader("Content-Type", "application/json")
    .withBody("{\"id\":101,\"name\":\"Phone\"}")));
given().port(8089).get("/products/101").then().statusCode(200).body("name", equalTo("Phone"));
wm.stop();
```

API Contract Testing (3)

88. How do you validate API contracts? What is API contract testing and why is it important? **[asked x3 in your bank]**

> **Answer:** Contract testing verifies that an API provider and its consumers agree on request and response expectations, so changes that break a consumer are caught early without full end-to-end tests.
>
> **Example:** The mobile app contract with the order API fails the build if a field it uses is removed.

89. What tools are used for contract testing? What is Pact and how does it work? **[asked x2 in your bank]**

> **Answer:** Tools: Pact, Spring Cloud Contract, Specmatic, Dredd, Postman schema checks. Pact is consumer-driven: the consumer test creates a pact file (contract), the provider verifies it, and a Pact Broker stores contracts and results.
>
> **Example:** Cart service (consumer) publishes its contract; the product service (provider) verifies it in CI.

90. What is the difference between schema validation and API contract testing?

> **Answer:** Schema validation checks that one response matches a structure. Contract testing checks the agreed interactions (request, response, status, headers, behaviour) between a specific consumer and provider and is driven by consumer needs.
>
> **Example:** Schema: price is a number. Contract: the cart service expects price and stock in /products/101.

Section 3: Medium Priority (28 questions)

Occasionally asked, usually as follow-ups or when your resume mentions the topic.

API Testing & HTTP Fundamentals (2)

91. What are the different types of APIs?

> **Answer:** An API (Application Programming Interface) lets one software talk to another: the client sends a request and the server returns a response. Types: REST, SOAP, GraphQL, gRPC; by access: public, partner, private and composite.
>
> **Example:** The mobile app calls the shop's REST API to get products and place orders.

92. What is the difference between functional and non-functional API testing?

> **Answer:** Functional testing checks correct behaviour (CRUD, validations, business rules). Non-functional checks performance, security, reliability, scalability and usability.
>
> **Example:** Functional: coupon reduces the total. Non-functional: coupon API handles 500 requests per second.

REST Assured: Requests, Specifications & Parameters (3)

93. How do you handle redirects in REST Assured?

> **Answer:** REST Assured follows redirects by default for GET; control it with redirects().follow(false) or redirects().max(n) and check the 3xx status and Location header.
>
> **Example:** Verify an old product URL redirects (301) to the new URL.

```
given().redirects().follow(false).when().get("/old-products/101")
  .then().statusCode(301).header("Location", containsString("/products/101"));
```

94. How do you configure relaxed HTTPS validation?

> **Answer:** Use relaxedHTTPSValidation() (or the global RestAssured.useRelaxedHTTPSValidation()) to ignore certificate errors on test environments only.
>
> **Example:** QA environment with a self-signed certificate.

```
given().relaxedHTTPSValidation()
  .when().get("https://qa.shop.com/health").then().statusCode(200);
```

95. How do you handle custom deserialization?

> **Answer:** When the default mapping is not enough use Jackson annotations (@JsonProperty, @JsonIgnoreProperties) or a custom deserializer registered in an ObjectMapper and pass it with ObjectMapperConfig.
>
> **Example:** Convert a date string like 31-10-2026 into a LocalDate field.

```
@JsonDeserialize(using = OrderDateDeserializer.class)
private LocalDate orderDate;
```

REST Assured: Response Validation & JSON Handling (3)

96. How do you parse XML responses and validate an XML schema? **[asked x3 in your bank]**

> **Answer:** For XML use xmlPath()/path expressions and hasXPath; for XSD validation use matchesXsdInClasspath.
>
> **Example:** Validate a legacy order XML response against order.xsd.

```
given().get("/legacy/orders/1001").then().statusCode(200)
  .body("order.id", equalTo("1001"))
  .body(matchesXsdInClasspath("schemas/order.xsd"));
```

97. How do you handle optional fields in JSON validation?

> **Answer:** Treat optional fields explicitly: assert them only when present (hasKey/nullValue) and mark only mandatory fields as required in the schema.
>
> **Example:** couponCode may be missing; the test passes with or without it.

```
Map<String, Object> body = res.jsonPath().getMap("$");
if (body.containsKey("couponCode")) Assert.assertNotNull(body.get("couponCode"));
```

98. How do you handle exceptions in REST Assured?

> **Answer:** A failed validation throws an AssertionError; handle connection and parsing problems (ConnectException, JsonPathException) with try-catch or by validating the status before parsing, and give clear failure messages.
>
> **Example:** If the API returns HTML on a 502, check the status first instead of failing on JSON parsing.

```
Response res = given().get("/orders/1001");
Assert.assertEquals(res.statusCode(), 200, "Unexpected status: " + res.asString());
int id = res.jsonPath().getInt("id");
```

Authentication, Security & Rate Limiting (1)

99. How do you test data encryption in API/DB?

> **Answer:** Check the API uses HTTPS/TLS (HTTP is rejected or redirected, HSTS header), sensitive fields are not returned in plain text and the database stores them encrypted/hashed.
>
> **Example:** Passwords and card numbers are never visible in responses or the DB in plain text.

```
Strict-Transport-Security: max-age=31536000
// Verify in the DB that password_hash and card_enc are not readable text
```

API Automation Framework Design (2)

100. How do you generate reports (Extent) for API tests? **[asked x2 in your bank]**

> **Answer:** Use ExtentReports with a TestNG listener, log request/response summaries and statuses for each test and generate an HTML report.
>
> **Example:** Report shows each API test with its request, response and status.

```
ExtentTest test = extent.createTest("Create order");
test.info("POST /orders");
test.pass("Status 201");   // test.fail(res.asString()) on failure
```

101. How do you version control API tests?

> **Answer:** Keep tests and collections in Git, work in feature branches with pull requests, add a .gitignore for build output and secrets, and tag releases.
>
> **Example:** Each new API test goes through a pull request review.

Postman Scripting & Newman (5)

102. What is a Monitor in Postman and how do you schedule collections? **[asked x2 in your bank]**

> **Answer:** A Monitor runs a collection on a schedule in the Postman cloud (from chosen regions) and alerts on failures by email or integrations. Create it from Monitors > Create, pick the collection, environment and schedule.
>
> **Example:** Run the shop health collection every 15 minutes in production.

103. How do you mock APIs in Postman?

> **Answer:** Create a mock server from a collection with saved example responses; use its mock URL in tests to simulate unavailable or failing services.
>
> **Example:** Mock the payment API to return a failure response for checkout negative tests.

104. How do you manage large collections and keep environments in sync across teams? **[asked x2 in your bank]**

> **Answer:** Use shared workspaces or version control, keep environment-specific values as variables, store secrets in current values or a vault (not shared), and sync via export/import or Git.
>
> **Example:** QA and dev share the QA environment file; each person keeps their own token locally.

105. What are common mistakes in Postman testing?

> **Answer:** Common mistakes: checking only status codes, hard-coded values, no negative tests, test order dependency, secrets in shared collections, not cleaning up data and not versioning collections.
>
> **Example:** A hard-coded orderId fails the next day because the order was deleted.

106. How do you run Postman collections in parallel?

> **Answer:** Newman runs one collection per process, so run several Newman processes at the same time (split by folder or collection) or use parallel stages in CI.
>
> **Example:** Cart and Orders folders run in parallel to cut regression time.

```
newman run shop.json --folder "Cart" &
newman run shop.json --folder "Orders" &
wait
```

CI/CD & Reporting for API Tests (1)

107. How do you integrate Postman with version control (Git)?

> **Answer:** Export the collection and environment JSON into a Git repository (or use Postman's Git/API integration) so changes are reviewed and Newman runs from the repo in CI.
>
> **Example:** Collection files stored in the repo and reviewed through pull requests.

Scenario-Based API Testing (7)

108. How do you test webhooks in Postman?

> **Answer:** Register a receiver URL (webhook.site, a mock server or an ngrok tunnel), trigger the event, then verify the callback arrives with the correct payload, signature/headers, retries and no duplicates.
>
> **Example:** Order shipped event posts to the registered URL with status SHIPPED.

109. How do you test GraphQL APIs in Postman?

> **Answer:** Send POST /graphql with a query and variables (Postman GraphQL body). Check the data and the errors array (status may still be 200), and test invalid fields, nested queries, mutations and auth.
>
> **Example:** Query a product with name, price and reviews in one call.

```
query ($id: ID!) { product(id: $id) { name price reviews { rating } } }
// variables: { "id": "101" }
```

110. How do you test APIs with file downloads?

> **Answer:** Send the GET request and check status, Content-Type, Content-Disposition and Content-Length, save the response to a file, compare size or checksum and test invalid ids, auth and large files.
>
> **Example:** Download invoice PDF for order 1001 and check the file is a valid PDF.

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="invoice-1001.pdf"
```

111. How do you test an API migration from monolith to microservices?

> **Answer:** Run the same requests against the monolith and the new services and compare responses (parity), test contracts, data consistency, gateway routing, auth propagation, failure handling and performance, with a rollback plan.
>
> **Example:** The order endpoint returns identical results before and after moving to the order microservice.

112. How do you test APIs with scheduled jobs?

> **Answer:** Trigger the job manually or with a short schedule in the test environment, wait/poll until it finishes, then verify its results, failure and retry handling, time zone and that it does not run twice.
>
> **Example:** Nightly job that cancels unpaid orders after 24 hours.

113. How do you test localization in APIs?

> **Answer:** Send the Accept-Language header and check translated messages, date/number/currency formats, default fallback for unsupported languages and UTF-8 characters in data.
>
> **Example:** Accept-Language: hi returns the error message in Hindi.

```
GET /products/101
Accept-Language: hi-IN
```

114. How do you test APIs with complex JSON structures?

> **Answer:** Break the response into parts and verify key fields, nested objects and arrays (size, types, each element), use JSONPath/jsonPath queries and a schema, and handle optional fields.
>
> **Example:** Check each item in order.items has sku, qty and price.

```
const items = pm.response.json().items;
pm.expect(items).to.be.an("array").that.is.not.empty;
items.forEach(i => pm.expect(i).to.have.all.keys("sku", "qty", "price"));
```

Reliability, Retry & Performance (1)

115. How do you perform load testing using REST Assured? How do you verify throughput and latency?

[asked x2 in your bank]

> **Answer:** REST Assured is not a load tool, but you can run small concurrent loads with an ExecutorService and measure response times; for real load and throughput/latency percentiles use JMeter, Gatling or k6.
>
> **Example:** Fire 50 parallel product-search calls and check the average time is below 500 ms.

```
ExecutorService pool = Executors.newFixedThreadPool(50);
List<Future<Long>> times = new ArrayList<>();
for (int i = 0; i < 50; i++)
  times.add(pool.submit(() -> given().get("/products?query=phone").getTime()));
pool.shutdown();
```

API Contract Testing (3)

116. What are the key components of an API contract?

> **Answer:** An API contract defines endpoints and methods, request parameters/headers/body, response schema, status codes, error format, authentication, versioning and constraints, usually written in OpenAPI.
>
> **Example:** OpenAPI file describing /orders request, 201 response and 400 error format.

117. How do you ensure backward compatibility in contract testing?

> **Answer:** Verify every new provider version against all existing consumer contracts (for example Pact can-i-deploy), allow additive changes only, deprecate gradually and run contract checks in CI.
>
> **Example:** Adding an optional field passes; removing a field used by the app fails.

118. What are the challenges of API contract testing?

> **Answer:** Challenges: maintaining contracts, many consumers, provider states and test data, learning the tools, cross-team coordination, versioning and setting up a broker.
>
> **Example:** Different teams need to agree on when to update shared contracts.

Section 4: Low Priority / Quick Revision (3 questions)

Niche protocols and rare scenarios. Know the one-line idea.

Authentication, Security & Rate Limiting (1)

119. How do you test APIs that require digital signatures?

> **Answer:** Create the signature (for example HMAC-SHA256) from the payload and secret in a pre-request script and send it in a header. Test valid signature, tampered body, wrong key, missing signature and replay with an old timestamp.
>
> **Example:** Payment callback must be rejected if the signature does not match.

```
const body = pm.request.body.raw;
const sig = CryptoJS.HmacSHA256(body, pm.environment.get("secret")).toString();
pm.request.headers.add({ key: "X-Signature", value: sig });
```

Scenario-Based API Testing (2)

120. How do you test gRPC APIs in Postman?

> **Answer:** In Postman create a gRPC request, enter the server URL, import the .proto file or use server reflection, select the method, send a JSON message and check the response and gRPC status (OK, NOT_FOUND, etc.); streaming calls are also supported.
>
> **Example:** Call the product service GetProduct method with id 101.

121. How do you test an API that returns streaming data?

> **Answer:** Use a client that supports streaming (curl -N for server-sent events, WebSocket tools), then verify chunks arrive in order with the right format, the connection stays open and closes properly, and reconnection works.
>
> **Example:** Live order-status updates sent as a stream.

```
curl -N -H "Accept: text/event-stream" https://api.shop.com/orders/1001/stream
```